

AGABI Backend Phase 2 — Implementation Blueprint · Version 1.1 (FROZEN)

Agabi Engineering

July 22, 2026

CONTENTS

AGABI Backend Phase 2 — Implementation Blueprint	
Version 1.1 — FROZEN	
What changed in v1.1	
1. Executive Summary	
1.1 What is being built	
1.2 Current state, verified	
1.3 The blocking precondition	
1.4 Shape of the program	
2. Phase 2 Constitution	
2.1 Purpose	
2.2 Mission	
2.3 Architectural Laws	
2.4 Engineering Laws	
2.5 Platform Invariants	
2.6 What Phase 2 will NEVER become	
3. Engineering Principles	
4. Explicit Non-Goals	
5. Build Philosophy	
5.1 Interfaces before implementations	
5.2 Vertical slice before breadth	
5.3 The falsifiable claim	
5.4 Content is engineering work	
6. Architecture Decision Log	
7. Architecture Boundaries	
8. Dependency Graph	
8.1 What can run in parallel	

8.2 What must be strictly serial

8.3 Circular dependency risks — checked and cleared

9. Critical Path

10. Build Order Justification

11. Complete Phase Breakdown

12. Phase Details

P2 · Preconditions

M0 · Knowledge spine — schema and store

M1 · Ingestion pipeline

M2 · Extractor and validation gates

M3 · Review CLI and the first verified chapter ⚠️ GATE

M4 · Search and path selection

M5 · Teaching bridge ⚠️ FIRST STUDENT-VISIBLE PHASE

M6 · Review UI and trust automation

M7 · Teaching Knowledge Layer ⚠️ GATE — the thesis test

M8 · Observation store

M9 · Assessment, breadth and releases

13. Milestones and Phase Exit Reviews

13.1 Milestones

13.2 Phase Exit Reviews

14. Testing Strategy

14.1 Three tiers

14.2 The golden set

14.3 What is deliberately not tested

15. Platform Success Metrics

15.1 Standing metrics

15.2 Review throughput — exact definition

15.3 Lesson quality — exact definition

16. Performance Budgets	
16.1 Latency — teach path (the only user-facing path)	
16.2 Latency — write and review paths	
16.3 Throughput	
16.4 Storage and memory	
16.5 Scale targets	
17. Operational Blueprint	
17.1 Deployment	
17.2 Monitoring	
17.3 Logging	
17.4 Backup and restore	
17.5 CI/CD	
17.6 Releases	
18. Failure Playbooks	
19. Migration Strategy	
19.1 Into the platform	
19.2 Schema evolution rules	
19.3 The fourteen migrations the architecture prevents	
20. Rollback Strategy	
21. Feature Flags and Kill Switches	
21.1 Flags	
21.2 Kill switches	
22. Risk Register	
23. Technical Debt Register	
24. Future Extension Points	
25. Architecture Validation Matrix	
26. Engineering Checklists	

- 26.1 Before starting any phase
- 26.2 Before opening a PR
- 26.3 Knowledge-integrity checklist (M3 onward)
- 26.4 Code review checklist
- 27. Definition of Done
- 28. Final Readiness Review
- 29. Build Order Summary
- 30. Appendices
 - A · Standing rules
 - B · Deferred decision defaults
 - C · Architecture traceability by phase
 - D · Glossary

AGABI Backend Phase 2 — Implementation Blueprint

Version 1.1 — FROZEN

- This document is frozen.** Together with Architecture Baseline v1.0 it forms the authoritative pair for Backend Phase 2. Implementation follows these two documents and nothing else.

Amendment clause. Changes require a written amendment recording: what changed, which section, which failure prompted it, and which test guards it. An undocumented edit to either document is architecture drift, not a correction.

Precedence. Where the two disagree, the Architecture Baseline wins and this blueprint is defective — report it, do not resolve it in code.

Authority: the frozen Architecture Baseline v1.0 is law. This document invents nothing. **Purpose:** architecture answers *what should exist*. This answers *exactly how we build it*. **Audience:** an engineer who has never seen this codebase. After reading, they should build Phase 2 without making a single architecture decision.

Convergence test: five senior engineers given only the Architecture Baseline and this document should produce implementations that converge. Where this document leaves a choice open, that is a defect in this document, not a decision for the engineer. Report it.

WHAT CHANGED IN V1.1

No architecture changed. v1.1 closes gaps where implementation still depended on engineering judgment.

Contradiction check: none found. One apparent conflict was investigated and resolved as consistent: `ClosureCache` (§7 M4) versus principle E5 *never store a conclusion*. The frozen architecture already draws this distinction — closures are classified **Derived · rebuildable · never authoritative** (`01-architecture.md:585`), and a `difficulty` column is forbidden in three separate places. A rebuildable cache is not a stored conclusion. Recorded as **ADR-11**; nothing escalated.

Seven ambiguities were found and closed. Each would have forced an engineer to stop and ask:

Gap in v1.0	Closed in v1.1
"lesson quality" at the M5/M7 gates was never defined — the gate could not fire	§15.3 scoring rubric, blind protocol, decision thresholds
"150 statements/hour" — measured how?	§15.2 exact definition
<code>TrustPolicy</code> shape unspecified, yet required on every content call	§7 M4 + ADR-6
M0's 16 tables had no stated creation order	§7 M0 FK-ordered list
M5 rollback "one line" — no flag specified	§21 <code>KNOWLEDGE_GROUNDING</code> flag + §21.2 kill switches
No kill switch for a runaway extractor	§21.2
Contradiction detection cadence unstated	ADR-9 — synchronous on promotion

New sections: **2** Constitution · **4** Non-Goals · **6** ADRs · **7** Boundaries · **10** Build Order Justification · **13.2** Exit Reviews · **15** Platform Metrics · **16** Performance Budgets · **17** Operational Blueprint · **18** Failure Playbooks · **22** Debt Register · **23** Extension Points · **24** Validation Matrix.

1. Executive Summary

1.1 What is being built

A knowledge platform that lets Agabi teach from **stored, verified knowledge** rather than from whatever a free-tier model invented at request time. Today `defaultOutline(topic)` is string templating — Agabi’s entire knowledge of photosynthesis is the word “photosynthesis”.

Phase 2 replaces one call site in `manager.ts` and puts a knowledge graph, a trust ladder, a teaching-asset layer and an observation store behind it.

1.2 Current state, verified

Read from the repository at commit `5e9ac15`:

Subsystem	State	Evidence
Phase 1 conversation architecture	complete	<code>advisors/</code> · <code>conversation/</code> · <code>evaluation/</code> , three walls enforced by <code>architecture.test.ts</code>
Trust boundary	complete	<code>advisors/advice.ts</code> — <code>Advice<T></code> carries <code>unknown</code> ; only exit is <code>accept()</code>
Evidence spine	complete	<code>evidence/{taxonomy,replay,outbox,failure}.ts</code> + tests · <code>health/</code> · <code>log.ts</code> · <code>Outbox</code> model
Lesson quality	complete	<code>conversation/quality.ts</code> + tests, <code>retryLesson</code>
Multi-canvas	code-complete, run-time-unverified	<code>canvasRepo.ts</code> , <code>ids.ts</code> , <code>/api/canvas/[canvasId]/teach</code> — DB lacks <code>Workspace.title/subject</code>
Knowledge platform	does not exist	no <code>knowledge/</code> , no <code>ingest/</code> , no <code>observation/</code>
Test suite	125 passing / 25 files	<code>npx vitest run</code>

1.3 The blocking precondition

`prisma db push` has never been run for `Workspace.title` and `Workspace.subject`. `setCanvasMeta` therefore throws on the first lesson of any canvas. **M0 does not begin until this is applied and multi-canvas is verified in a browser.**

Not a formality. Building a knowledge platform on top of a feature nobody has seen execute is how a whole phase gets built against a broken assumption.

1.4 Shape of the program

Ten phases, M0–M9. ~13–15 weeks engineering. Content review dominates the calendar and is the schedule risk, not the code.

Two hard gates where the program **stops and reports a number** before continuing:

- **After M3** — measured review throughput (§15.2). Below 150 statements/hour, M6 moves ahead of M5.
- **After M7** — grounded vs asset-supported lesson quality (§15.3). If both M5 and M7 are flat, the central thesis is falsified and M9’s 40–80 hours of content work must not be spent.

2. Phase 2 Constitution

Permanent. Amendable only by written amendment recording what changed, which section, which failure prompted it, and which test guards it.

2.1 Purpose

Agabi teaches from knowledge it can justify. Every claim a student sees traces to a source a human verified.

2.2 Mission

Build the substrate that makes justified teaching possible: a knowledge graph, a trust ladder, a teaching-asset layer, an observation store — none of which the student ever sees directly, all of which determine what the student receives.

2.3 Architectural Laws

Violating one of these is not a bug. It is a breach of the design.

L1 Three graphs stay separate. `REQUIRES` and `PART_OF` are acyclic; reinforcement may cycle. No unified `Edge` table, ever. **L2** Nothing above `AUTO_VALIDATED` exists without a human `ReviewEvent`. **L3** Every statement above `MACHINE_PROPOSED` carries provenance with a machine-checked quote. **L4** Identity is opaque and immutable. Slugs move; ids do not. **L5** Knowledge is append-only. There is no delete path. **L6** Knowledge and observation live in separate stores and are never joined. **L7** Curriculum is a mapping layer. Knowledge never references it. **L8** Conclusions are derived, never stored. No difficulty, no mastery score, no quality rating. **L9** Prisma exists only inside `store/**`. **L10** `Provenance.quote` is verification evidence and is never served to a learner.

2.4 Engineering Laws

G1 Every phase leaves the repository shippable. **G2** No phase weakens an earlier phase's test. **G3** Every phase traces to a named architecture section. **G4** Rollback exists before merge. **G5** Deferred decisions take their default (§34/Appendix B) and record an observation. **G6** No new npm dependency in M0–M9. **G7** Every `prisma db push` is gated on a human.

2.5 Platform Invariants

The standing tests in §14.1. Deleting one abandons the principle it guards, regardless of what any document says.

2.6 What Phase 2 will NEVER become

- A model that answers from its weights and calls it knowledge
 - A system where a machine promotes its own output to verified
 - A recommendation engine
 - A place where a learner's data and the knowledge graph share a query
 - A schema with a difficulty column
 - A curriculum-shaped graph that must be rebuilt per board
-

3. Engineering Principles

Binding. Violating one is grounds for rejecting a PR regardless of whether tests pass.

E1 · Architecture is law. If this blueprint and the architecture disagree, the architecture wins and the blueprint is defective. Report it; do not resolve it in code.

E2 · Every phase leaves the repository shippable. `tsc --noEmit`, `lint`, `vitest run`, `build` all clean.

E3 · Tests are invariants over data, not exercises of code. `dependency-dag` asserts a property of the graph, forever. These are the load-bearing tests.

E4 · No phase weakens an earlier phase's test. If a new phase makes an old test fail, the new phase is wrong until proven otherwise.

E5 · Never store a conclusion. If it can be recomputed, compute it on read. Derived caches are permitted only when rebuildable, invalidatable, and never authoritative (ADR-11).

E6 · The store is the only place Prisma appears. A leak here is how engine independence dies.

E7 · Deferred decisions take their default. Take it, record the observation, do not deliberate.

E8 · Stop and ask before `prisma db push` **or any new dependency.**

E9 · Rollback must exist before merge.

4. Explicit Non-Goals

Phase 2 deliberately does not solve these. Each is listed with the extension point that will carry it later (§23), so deferring is not the same as forgetting.

Not built in Phase 2	Why deferred	Lands via
Digital Twin	needs longitudinal observations that only exist after M8 runs in production	X6
Mastery Engine	<code>mastery()</code> in M8 is a pure query with no decay, no forgetting curve, no inference	X4
Recommendation	requires efficacy data across many learners; premature at $n \approx 1$	X5
Memory / personalization	Phase 3. <code>selectPath</code> in M5 takes no learner input by design	X4
Adaptive teaching	asset selection in M7 is trust + context + recency; no adaptation loop	X3
AI Tutor / dialogue	Phase 1 conversation is unchanged; Phase 2 changes what it teaches, not how it talks	—
Community contribution	D8 defaults to single reviewer until contributors > 1	X2
Vector / semantic search	rungs 1–3 only. D9 holds rung 4 until rung 3 measurably fails	X1
Multi-tenancy beyond isolation tests	<code>tenant-isolation</code> test exists; no tenant UI, billing, or quota	—
Real-time collaborative review	M6 is single-reviewer-at-a-time; no locking, no presence	X2
Content authoring UI	reviewers edit proposals; they do not write from scratch	X2
Analytics dashboards	evidence spine emits; nothing consumes it visually	X7

Scope-creep rule. If a proposed change is on this list, it is out of Phase 2 regardless of how small it looks. Small versions of these are the expensive ones — they establish schema that Phase 3 must then unwind.

5. Build Philosophy

5.1 Interfaces before implementations

`KnowledgeStore` is written and its conformance suite passes against an **in-memory implementation first**, before Postgres. Writing the memory implementation first is what forces the interface to be storage-agnostic. An interface designed against Prisma contains Prisma-shaped assumptions nobody notices until a second implementation is attempted.

5.2 Vertical slice before breadth

M0–M5 build one thin vertical path: schema → ingest → extract → review → search → a student sees a grounded lesson. Only then does M6 widen review, M7 add teaching assets, M9 add breadth.

The alternative — all the platform, then all the content — means the first student-visible result arrives after every expensive decision is already sunk.

5.3 The falsifiable claim

The program asserts: **grounding alone produces a small accuracy gain and no perceived-quality gain; the large gain arrives with teaching assets**. Stated in advance, measured at M5 and M7 by the rubric in §15.3. If both are flat, the thesis is wrong and the program stops.

A program that cannot be falsified is not engineering.

5.4 Content is engineering work

40–80 hours of human review is a line item. Every design choice in M3 and M6 optimises review throughput, because the bottleneck is the ratio of human attention to justified knowledge — not the schema.

6. Architecture Decision Log

Each ADR records a decision already made in the frozen architecture. Purpose: preserve intent so a future engineer does not re-litigate it from ignorance.

ADR-1 · Three separate graphs, not one edge table

Decision. `DependencyEdge`, `CompositionEdge`, `ReinforcementEdge` are separate tables with separate invariants.

Alternatives. (a) One `Edge` table with a `kind` column. (b) Two graphs, folding composition into dependency.

Rejected because. A unified table cannot express that `REQUIRES` must be acyclic while reinforcement must be allowed to cycle — the acyclicity test would have to be conditional on a column value, which means one bad row silently disables it. Folding composition into dependency conflates “you must learn A first” with “A is part of B”, which are different teaching operations: one orders a path, the other decomposes a topic.

Consequences. Three DAG tests instead of one. `graph-conflict` needed to prevent the same ordered pair appearing as both a prerequisite and a reinforcement. Import rules W6/W7 keep the modules apart.

Future. A fourth graph is a new table plus a new invariant test, not a migration.

ADR-2 · Trust is a ladder with a human floor

Decision. Six levels. Promotion above `AUTO_VALIDATED` requires a `ReviewEvent` with a human `actorId`.

Alternatives. (a) `verified: boolean`. (b) A numeric confidence score. (c) Model self-assessment.

Rejected because. A boolean cannot express “all validators passed, no human has looked” — a real and useful state that describes most of the graph most of the time. A numeric score invites threshold tuning and hides *why* something is trusted. Model self-assessment is the failure this entire phase exists to prevent: an extractor that can mark its own output verified has no trust boundary at all.

Consequences. `RawProposal` deliberately has no trust field. Trust is assigned by the platform, never claimed by the producer.

Future. New levels insert into the ladder; `TrustPolicy` shields callers from the change.

ADR-3 · Grounding is exact containment, never fuzzy

Decision. V3 checks that the quote appears literally in the chunk after normalisation, and returns the character range.

Alternatives. (a) Fuzzy/similarity match above a threshold. (b) Model-judged grounding.

Rejected because. Fuzzy matching admits paraphrase-as-quote, which is precisely the fabrication mode V3 exists to catch — the extractor produces something that *reads* grounded and is not. Model-judged grounding asks the fabricator to grade itself.

Consequences. Extraction prompts must instruct exact copying. Some legitimate proposals fail V3 and need re-extraction. Accepted: false negatives are cheap, false positives are unrecoverable.

Future. The returned char range is what M3 and M6 highlight. Any change here breaks review ergonomics.

ADR-4 · Opaque immutable ids, mutable slugs

Decision. `Concept.id` is cuid2, generated in-process, never by a database default. No FK targets a slug.

Alternatives. (a) Slug as primary key. (b) Auto-increment integer. (c) DB-generated uuid.

Rejected because. Slugs are editorial and change when a concept is renamed or disambiguated; a slug PK turns a rename into a cascading migration. Auto-increment leaks ordering and volume, and collides on merge across environments. DB-generated ids cannot be assigned before insert, which breaks batch construction of a graph where edges reference nodes not yet written.

Consequences. `identity` test asserts no FK targets a slug and no id is ever updated. Merge (§7 M6) is expressible because the loser id keeps resolving forever.

ADR-5 · Append-only, no delete path

Decision. `KnowledgeStore` exposes no delete method. Absence is the enforcement.

Alternatives. (a) Soft delete flag. (b) Hard delete with audit log.

Rejected because. A soft-delete flag must be honoured by every read path forever; one forgotten `WHERE deleted = false` silently resurrects retracted knowledge. Hard delete makes point-in-time replay impossible — a lesson taught in March cannot be reconstructed if its statements are gone.

Consequences. Retraction is demotion plus a `ReviewEvent`. Storage grows monotonically; accepted (§16.4). The `no-delete` test scans for destructive paths under `knowledge/`.

Exception. Learner erasure under DPDP deletes from the **observation** store only (L6 makes this safe).

ADR-6 · `TrustPolicy` is a required parameter with no default

Decision. Every content-returning store call takes an explicit `TrustPolicy`.

```
type TrustPolicy = {
  minimum: TrustLevel      // inclusive floor on the ladder
  allowDisputed: false     // literal false — DISPUTED is never servable
  requireProvenance: boolean // default true at call sites that teach
}
```

Alternatives. (a) Default to `VERIFIED`. (b) Global config. (c) Per-call boolean `includeUnverified`.

Rejected because. A default means a new call site silently inherits a trust decision its author never considered — and the failure mode is serving `MACHINE_PROPOSED` content to a child. Global config makes the trust level invisible at the call site, which is exactly where a reviewer needs to see it. A boolean cannot express a ladder.

Consequences. Verbose call sites, deliberately. Omitting the parameter fails to compile — an invariant enforced by the type system rather than a test.

ADR-7 · One call site changes in M5

Decision. Only `startLesson` in `manager.ts` changes. `repairOutline`, `buildSkeleton`, `coerceSlot`, `adaptBlock`, `fill-Chunk` and every renderer are untouched.

Alternatives. (a) Grounding-aware block generation throughout. (b) A parallel grounded lesson pipeline.

Rejected because. Threading knowledge through the whole pipeline makes rollback a multi-file revert under pressure, and couples two subsystems that the architecture deliberately separated. A parallel pipeline doubles the surface that must satisfy the visual guarantees, and the two would drift.

Consequences. Rollback is one line. If a second call site appears to need changing, something is wrong — stop and re-read §8.2.

ADR-8 · `DISPUTED` is a flag, not a rung

Decision. Dispute suspends servability while preserving the trust level underneath.

Alternatives. Make `DISPUTED` a level on the ladder.

Rejected because. As a level it erases what the statement was trusted at before the dispute. Resolution would then require full re-verification from scratch rather than one decision, and a statement verified by three reviewers would land in the same bucket as one that was never examined.

Consequences. `TrustPolicy.allowDisputed` is literal `false`. Resolution restores prior standing.

ADR-9 · Contradiction detection runs synchronously on promotion

Decision. Contradiction check executes inside the promotion transaction. It does not run as a batch job.

Alternatives. (a) Nightly batch. (b) Async queue.

Rejected because. Batch and async both open a window where a contradicted statement is servable at `VERIFIED`. Since contradiction *blocks* promotion (§14.3), running it after promotion inverts the control.

Consequences. Promotion latency includes the check — budgeted at §16.2. On an empty graph it finds nothing and costs nothing, which is how M3 bootstraps.

ADR-10 · Never auto-merge concepts

Decision. Similarity ≥ 0.85 queues a merge for human decision. Nothing merges automatically.

Alternatives. Auto-merge above a high threshold.

Rejected because. A wrong merge is worse than a duplicate and is silent. A duplicate is visible and annoying; a wrong merge conflates two ideas, and every observation, edge and asset downstream inherits the error with no signal that anything happened.

Consequences. A merge queue that a human must service. Duplicate rate is a tracked KPI (§15.1) rather than a solved problem.

ADR-11 · Derived caches are permitted; stored conclusions are not

Decision. `ClosureCache` and search indexes are allowed. Difficulty, mastery and efficacy columns are not.

Rationale. The frozen architecture already draws this line: derived artefacts are **rebuildable · never authoritative** (`01-architecture.md:585`). A cache can be dropped and recomputed identically from source data. A stored conclusion cannot — it encodes a judgment made by a model version that no longer exists, and improving the model would invalidate history instead of reinterpreting it.

Test. Anything cached must have a rebuild path that produces byte-identical output. If it cannot be rebuilt, it is a conclusion, not a cache.

ADR-12 · Postgres for traversal, until measured otherwise

Decision. Recursive CTE plus `ClosureCache`. No graph database.

Rejected because. `REQUIRES` closure is single-digit depth by cognitive necessity — a prerequisite chain deeper than about six is a curriculum bug, not a traversal problem. The rest of the workload (context ranking, trust filtering, text matching) is relational. Adding a second datastore for a bounded traversal buys latency at the cost of consistency and operational surface.

Falsifier. p95 closure > 50 ms **with** the cache warm → introduce a traversal engine as a derived store behind the same interface (D10). Not before.

7. Architecture Boundaries

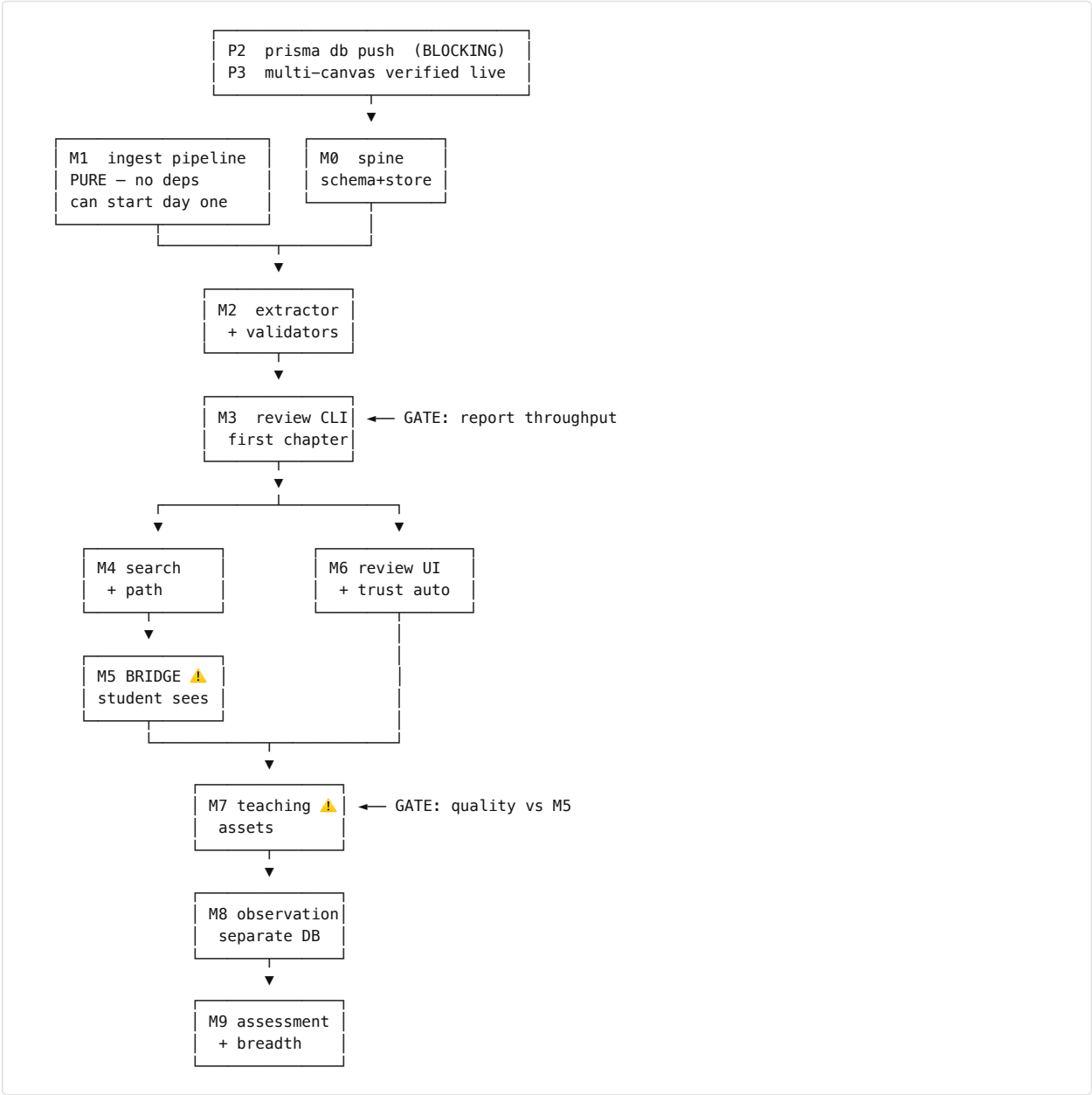
For each subsystem: what it owns, what it must not own, what it consumes and produces, and which imports are forbidden. Ambiguity here is what causes architecture drift.

Subsystem	Owns	Does NOT own	Consumes	Produces	Forbidden imports
<code>ingest/</code>	parsing, cleaning, normalising, chunking, span arithmetic	storage, extraction, meaning	raw bytes, URLs	<code>Source</code> , <code>SourceChunk</code> , <code>Span[]</code>	<code>knowledge/store/**</code> , <code>@prisma/client</code> , <code>advisors/**</code>
<code>advisors/knowledge/</code>	prompts, schemas, <code>PROMPT_VERSION</code> , model calls	trust, validation verdicts, persistence	<code>SourceChunk</code>	<code>Advice<RawProposal[]></code>	<code>knowledge/store/**</code> , <code>@prisma/client</code>
<code>knowledge/trust/</code>	validators V1–V15, ladder, policy, promotion, demotion, corroboration, contradiction	model calls, storage, review UI	<code>RawProposal</code> , existing statements	verdicts, <code>TrustLevel</code> , <code>Contradiction</code>	<code>advisors/**</code> , <code>@prisma/client</code>
<code>knowledge/graph/</code>	three edge kinds, traversal, topological sort, cycle detection	trust, teaching, curriculum	edges, <code>TraversalSpec</code>	ordered closures	<code>@prisma/client</code> ; <code>dependency.ts</code> ↗ <code>reinforcement.ts</code> (W6/W7)
<code>knowledge/store/</code>	the only Prisma surface , conformance contract	domain logic, validation, ordering policy	domain objects	persisted rows	<code>advisors/**</code> , <code>conversation/**</code>
<code>knowledge/teaching/</code>	asset kinds, capability registry, <code>as-setsFor</code>	assessment, efficacy computation, lesson composition	concepts, context, trust policy	<code>TeachingAsset[]</code>	<code>@prisma/client</code> , <code>observation/**</code>
<code>knowledge/assessment/</code>	items, distractors, misconception	scoring,	concepts	<code>AssessmentItem[]</code>	<code>observation/**</code> , <code>@prisma/client</code>

Subsystem	Owns	Does NOT own	Consumes	Produces	Forbidden imports
	diagnosis	cali- bra- tion, diffi- culty			
<code>observation/</code>	<code>Observation</code> <code>rows</code> , <code>mastery()</code> , <code>efficacy()</code> , <code>earn()</code> , <code>purge-</code> <code>User</code>	knowl- edge, con- cepts, assets (refer- ences by id only)	events, ids	derived metrics, earned edges	<code>knowledge/store/**</code> — any join across stores is a test failure
<code>review/</code>	queue ordering, batching, atomic commit, merge/split	trust rules, pro- mo- tion au- thority	proposals, contradictions	<code>ReviewEvent</code>	<code>advisors/**</code>
<code>conversation/</code> (Phase 1)	lesson shape, blocks, repair, rendering	knowl- edge, trust, stor- age of knowl- edge	outline (grounded or default)	lesson blocks	<code>knowledge/store/**</code> — reads only via <code>knowledge/path.ts</code>

The single crossing point. `conversation/manager.ts` → `knowledge/path.ts`. That is the whole integration surface between Phase 1 and Phase 2 (ADR-7). Any second crossing is architecture drift.

8. Dependency Graph



8.1 What can run in parallel

Parallel	Why safe
M1 alongside M0	M1 is pure functions with no store dependency. Different engineer, different files, zero overlap.
M4 alongside M6	M4 touches <code>search.ts</code> / <code>path.ts</code> ; M6 touches <code>trust/</code> and review routes. Both depend on M3, neither on the other.
Golden-set authoring alongside M2	content work, not code

8.2 What must be strictly serial

Serial	Why
M0 → M2	the extractor's output must be typed against real schema
M2 → M3	nothing to review until proposals exist
M3 → M4	search and path selection are untestable against an empty graph
M4 → M5	the bridge needs <code>selectPath</code>
M7 → M8	efficacy needs assets to measure

8.3 Circular dependency risks — checked and cleared

Suspected cycle	Resolution
trust needs contradiction detection; contradiction needs verified statements	Not circular. Contradiction detection reads whatever exists; on an empty graph it finds nothing and every statement passes that gate. Bootstraps naturally.
<code>assetsFor</code> needs efficacy; efficacy needs observations of assets	Not circular. Efficacy returns <code>INSUFFICIENT_DATA</code> ; selection falls back to trust level then recency.
<code>selectPath</code> needs mastery; mastery needs observations from lessons	Not circular. M5's <code>selectPath</code> takes no mastery input. Phase 3 adds it as a filter step.
review reputation needs review history	Not circular. M3 is single-reviewer; reputation is inert until M6 and D4 defaults to expert-only.

No circular implementation dependency exists. Every apparent one bootstraps from an empty state with defined behaviour.

9. Critical Path

P2 → M0 → M2 → M3 → M4 → M5

M1 is off the critical path — parallel from day one, never a blocker.

M0 is the highest-leverage phase. Everything writes through `KnowledgeStore`. An interface defect propagates into every subsequent phase and is expensive to unwind after M2–M5 depend on it. Spend disproportionate review effort here.

M3 is the highest-uncertainty phase. It produces the first real measurement of review throughput, the program's actual bottleneck.

The bottleneck is not code. It is human verification capacity. Engineering effort should go to anything that reduces human attention per justified statement: validators, corroboration, contradiction detection, batch ergonomics.

10. Build Order Justification

Why this order, and what breaks under the plausible alternatives.

Position	Why here	If moved earlier	If moved later
P2 first	multi-canvas is unverified; everything downstream assumes it works	n/a	a whole phase built on a feature that throws on first use
M0 before M2	proposals must be typed against real schema	n/a	extractor output shaped by guesswork, retyped later — rewrite of M2
M1 parallel, not serial	pure functions, zero shared files with M0	n/a	1 week added to critical path for no benefit
M2 before M3	nothing to review until proposals exist	reviewers idle	—
M3 before M4	search and ordering are untestable on an empty graph; and M3 yields the throughput number that may reorder the rest	tests pass against nothing, which proves nothing	the bottleneck stays unmeasured deeper into the program
M4 before M5	the bridge needs <code>selectPath</code>	bridge has nothing to call	—
M5 before M6	first student-visible result; validates the thesis before more platform is built	—	the classic failure: platform complete, product never shipped (IR1)
M6 parallel with M4	disjoint files, shared dependency on M3 only	—	review stays slow while content demand grows
M7 after M6	asset review needs the widened review pipeline	assets reviewed through the M3 CLI at 1/3 the rate	—
M8 after M7	efficacy has nothing to measure without assets	<code>AssetEfficacy</code> rows with no assets	—
M9 last	40–80 content hours must not be spent before the M7 gate confirms the thesis	the single most expensive irreversible commitment made on an unvalidated premise	—

The two orderings that matter most. M5 before M6 is what prevents the program from becoming a platform nobody uses. M9 after the M7 gate is what prevents 80 hours being spent on a falsified thesis.

The one permitted reordering. If M3 reports throughput below 150/hr, M6 moves ahead of M5 (§13.2). This is not a schedule slip — it is the plan responding to its own measurement.

11. Complete Phase Breakdown

#	Phase	Depends on	Parallel with	Eng	Content	Gate
P2	<code>db push</code> + live verification	—	—	1 day	—	blocking
M0	Knowledge spine: schema + store	P2	M1	2–3 wk	—	
M1	Ingestion pipeline (pure)	—	M0	1 wk	—	
M2	Extractor + validation gates	M0, M1	—	1–2 wk	—	
M3	Review CLI + first chapter	M2	—	1 wk	4–6 hr	throughput
M4	Search + path selection	M3	M6	1 wk	—	
M5	Teaching bridge	M4	—	1 wk	—	student-visible
M6	Review UI + trust automation	M3	M4	2–3 wk	—	
M7	Teaching Knowledge Layer	M6	—	2 wk	6–10 hr	quality
M8	Observation store	M7	—	1 wk	—	
M9	Assessment + breadth + releases	M8	—	2 wk	40–80 hr	

12. Phase Details

P2 · Preconditions

Purpose. Make multi-canvas actually run before building on it.

Architectural objective. None — debt repayment. Architecture §21.1 assumes a working base.

Steps.

1. Tanveer runs `npx prisma db push` (E8/G7 — the classifier blocks the agent).
2. `npx prisma generate`.
3. Browser verification: type a topic on `/` → lands on `/c/{id}` → lesson streams → refresh resumes the same canvas → open a second canvas → confirm the AI has no memory of the first.

Failure modes. Push reports destructive changes → **stop**, report, do not proceed. `setCanvasMeta` still throws → schema and client out of sync; run `prisma generate`.

Rollback. Columns are nullable and additive; nothing to undo.

Exit criteria. A canvas's first lesson completes without an exception, and two canvases are demonstrably isolated.

M0 · Knowledge spine — schema and store

Purpose. Every later phase writes through this. Get the interface right or pay for it ten times.

Architectural objective. §10 (data model), §9 (modules), §17 (storage), §18A (identity), §18B (traversal), §11 (three graphs).

Files created.

```
src/server/knowledge/ids.ts
src/server/knowledge/concept.ts
src/server/knowledge/statement.ts
src/server/knowledge/context/{registry,canonical,match}.ts
src/server/knowledge/graph/{dependency,composition,reinforcement,traverse}.ts
src/server/knowledge/store/KnowledgeStore.ts
src/server/knowledge/store/memory.ts           ← WRITE THIS FIRST (§5.1)
src/server/knowledge/store/postgres.ts
src/server/knowledge/store/conformance.test.ts
src/server/knowledge/version.ts
```

Files modified. `prisma/schema.prisma` · `src/server/architecture.test.ts` (W2–W7).

Database changes — one gated `db push`, tables created in FK order:

1. Source	
2. SourceChunk	→ Source
3. ContextDimension	
4. Context	→ ContextDimension
5. Concept	
6. ConceptAlias	→ Concept
7. ConceptTag	→ Concept
8. DependencyEdge	→ Concept ×2
9. CompositionEdge	→ Concept ×2
10. ReinforcementEdge	→ Concept ×2
11. Statement	→ Concept, Context
12. Provenance	→ Statement, SourceChunk
13. Contradiction	→ Statement ×2
14. ReviewEvent	→ Statement
15. Release	
16. ReleaseMember	→ Release, Statement

Order matters: Prisma emits them as written, and a forward reference fails the push mid-way, leaving a partial schema (§18 play-book FP-1).

Interfaces introduced. KnowledgeStore (§17), TraversalSpec (§18B.1), Context + ContextDimension registry (§18).

Implementation notes, binding.

- Concept.id is cuid2, generated in-process, **never** by a database default (ADR-4).
- Context.id = sha256(canonicalJSON(dimensions)), keys lexicographically sorted, values normalised per dimension type. Get this wrong and identical contexts fragment silently.
- traverse.ts requires maxDepth and maxNodes with **no defaults**. Truncation returns truncated: true.
- Topological sort lives in dependency.ts only. W6 forbids it importing reinforcement.ts.
- KnowledgeStore exposes **no delete method** (ADR-5).

Tests required.

Test	Asserts
conformance	both store implementations satisfy one suite
dependency-dag	REQUIRES acyclic, whole graph
composition-dag	PART_OF acyclic
reinforcement-cycles-pass	a deliberate cycle does not fail
graph-conflict	no ordered pair in both dependency and reinforcement
context-canonical	key order irrelevant; identical dimension sets hash identically
identity	no FK targets slug; Concept.id never updated
no-delete	no destructive path under knowledge/
empty-platform	every method returns sensible empties on zero rows
walls	W1–W7
traversal-bounded	omitting maxDepth fails to compile; a cyclic fixture terminates

Integration tests. None — M0 has no consumer yet. Correct and expected.

Failure modes. Interface leaks Prisma types → W5 fails. Cuid2 generated at DB level → identity fails. Context hashing non-deterministic → context-canonical fails.

Rollback. Drop the new tables. No existing code references them.

Acceptance criteria. Conformance suite green against both implementations. A cycle inserted into `ReinforcementEdge` passes CI; the same cycle in `DependencyEdge` fails with the cycle path printed.

Definition of Done. §26 plus: all tests green, `architecture.test.ts` extended, one commit.

M1 · Ingestion pipeline

Purpose. A PDF becomes deterministic, content-addressed, locator-preserving chunks.

Architectural objective. §12.1–12.3.

Files created.

```
src/server/ingest/spans.ts           ← Span[]; offsets survive every transform
src/server/ingest/parse/{pdf,markdown,html}.ts
src/server/ingest/{clean,normalise,chunk,pipeline}.ts
```

Database changes. None.

Implementation notes, binding.

- Text is carried as `Span[]`, **never** as a plain string. Removing a header drops a span; other offsets stay intact.
- `chunkId = sha256(sourceId + JSON(locator) + normalisedText)`.
- Every stage is a pure function. No I/O except `fetch`.

Tests required. Determinism (ingest twice, byte-compare all chunk ids) · span preservation (a known quote's char range still maps to the right page after clean + normalise) · cleaning does not over-strip (fixture with headers, footers, exercise numbering) · re-ingestion diff (edit one paragraph → exactly one chunk id changes).

Failure modes. Locator loss is unrecoverable — a statement whose source position was lost can never be verified. The span-preservation test is the only defence.

Rollback. Delete the directory. Nothing imports it yet.

Acceptance criteria. One real NCERT chapter produces stable chunks across runs; a one-paragraph edit produces exactly one new chunk id.

M2 · Extractor and validation gates

Purpose. A chunk produces validated proposals. Nothing reaches the graph.

Architectural objective. §8.1 (trust boundary), §12.4 (four passes), §14 (V1–V15).

Files created.

```
src/server/advisors/knowledge/extractEntities.ts
src/server/advisors/knowledge/extractStatements.ts
src/server/advisors/knowledge/extractDependencies.ts
src/server/advisors/knowledge/{prompts,schemas}.ts ← PROMPT_VERSION here
src/server/knowledge/trust/{ladder,policy}.ts
src/server/knowledge/trust/validators/{grounding,payload,units,dates,refs,scope,originality}.ts
```

Implementation notes, binding.

- The extractor returns `Advice<RawProposal[]>`. `RawProposal` **has no trust field** (ADR-2).
- V3 grounding is **exact containment after normalisation, never fuzzy** (ADR-3). Returns the char range, which M3 and M6 need for highlighting.
- Pass 3 classifies each dependency as `REQUIRES` / `PART_OF` / reinforcement and is **always surfaced for human confirmation** regardless of trust (V9). This is what keeps the DAG honest.
- D1 default: four passes.

Tests required. V1–V15 each with a passing and a failing fixture · **the fabricated-quote fixture** (“photosynthesis releases nitrogen” — grounded-looking, quote absent from source) must be rejected with zero human involvement · `trust-gate` · golden-set scoring harness (records scores, does not yet gate).

Failure modes. Fuzzy grounding admits paraphrase-as-quote. Extractor returns valid JSON with a hallucinated quote → V3 catches it. >50% batch failure → halt the source (§12.5, playbook FP-4).

Rollback. Disable the pipeline entry point. Validators are pure and inert.

Acceptance criteria. One chapter produces proposals; every one carries a quote literally present in its chunk; a hand-injected hallucination is rejected automatically.

Definition of Done. §26 plus: `PROMPT_VERSION` set, golden set scored, number recorded.

M3 · Review CLI and the first verified chapter ⚠️ GATE

Purpose. A human promotes proposals to `OFFICIAL_SOURCE_VERIFIED`. First real knowledge exists. First real throughput measurement.

Architectural objective. §25 (review), §26.2 (promotion), §19 (versioning).

Files created.

```
src/server/review/{queue,batch,decide}.ts
src/server/knowledge/review.ts
scripts/review-cli.mjs      ← source pane + proposals + highlighted quote
```

Implementation notes, binding.

- **Promotion above** `AUTO_VALIDATED` **requires a** `ReviewEvent` **with a human** `actorId` (L2). No flag, no threshold, no override.
- Batch decisions commit atomically. A mid-batch failure commits nothing.
- The CLI shows the source passage with the quote highlighted in place, **≤8 proposals per screen** (a hard limit, not config — §25.2 ties it to reviewer attention), approve-all as one keystroke.

Tests required. `trust-gate` · `provenance` · batch atomicity · `no-silent-uncertainty`.

Content. NCERT Class 10 Science, Life Processes. Target ~150 concepts, ~400 statements at `OFFICIAL_SOURCE_VERIFIED`.

Author the golden set for the same chapter while the source is already open.

Failure modes. Reviewer fatigue producing rubber-stamped approvals — mitigated by batch size ≤8 and leverage ordering. Throughput below target — this is the gate.

Rollback. Promotions are events; demote by appending events. Nothing is lost.

⚠️ **GATE.** Throughput per §15.2.

- **≥150/hr** → proceed to M4.

- **<150/hr → build M6 before M5.** The tooling is the bottleneck; widening it first is cheaper than discovering it after the bridge is built.

Acceptance criteria. One chapter fully verified, golden set authored, throughput number reported.

M4 · Search and path selection

Purpose. `resolve("photosynthesis")` returns concepts; `selectPath` returns a prerequisite-ordered plan.

Architectural objective. §15 (search), §16 (API), §18B.3 (sort), §18 (indexing).

Files created. `src/server/knowledge/{search,path,curriculum}.ts`

Database changes. `Program`, `ProgramNode`, `Mapping`, `LearningObjective`, `ObjectiveConcept`, `ClosureCache`. **Gated push.**

Implementation notes, binding.

- `TrustPolicy` is a **required parameter with no default** on every content-returning call (ADR-6).
- `selectPath` is deterministic — no model call. Tie-break by `Mapping.ordinal`, else concept id.
- Rungs 1–2 only. Rung 3 (trigram) lands in M6; rung 4 is D9.
- Closure cache invalidation: **clear all on review commit** (D3 default). Crude and correct; §16.2 budgets the rebuild.

Tests required. `curriculum-independence` (drop all program rows, `selectPath` still works) · path determinism (snapshot) · `trustPolicy` omission fails to compile · closure cache invalidates on edge write · **cache rebuild produces byte-identical output** (ADR-11).

Failure modes. Non-deterministic sort → replay breaks. Cache staleness → stale prerequisites; mitigated by clear-all.

Rollback. Unused by teaching until M5.

Acceptance criteria. Prerequisites before core, topologically ordered, budget-bounded, byte-identical across runs.

M5 · Teaching bridge 🚨 FIRST STUDENT-VISIBLE PHASE

Purpose. A student typing “photosynthesis” receives a lesson where every block traces to a concept id.

Architectural objective. §8.2 — the one call site (ADR-7).

Files modified. `src/server/conversation/manager.ts` (`startLesson` only) · `src/server/conversation/prompt.ts` (grounded prompt; bump `PROMPT_VERSION`) · `src/server/knowledge/path.ts` (`outlineFrom`).

Database changes. None.

Implementation notes, binding.

- **Exactly one call site changes.** If a second appears to need changing, stop.
- Gated by feature flag `KNOWLEDGE_GROUNDING` (§21.1). Default off until the acceptance comparison is run.
- Empty path → `defaultOutline(topic)` + `knowledge.miss` event. A supported state, not an error.
- The evidence spine records `grounded`, concept ids, versions, release id per lesson.

Tests required. Grounded end-to-end (every block carries a concept id) · fallback emits `knowledge.miss` · `repairOutline` invariants still hold on grounded outlines — heading bookend, three-visual floor, max text run · evidence stamping · flag off ⇒ byte-identical to pre-M5 behaviour.

Integration test. Teach the verified chapter's topic; assert every emitted block resolves to a `VERIFIED` statement.

Failure modes. Grounded outlines violating the visual guarantee → `repairOutline` catches it, but the test must prove it. Latency regression from graph reads → §16.1 budget.

Rollback. Flag off, or one line — revert `startLesson` to `defaultOutline(topic)`. **This property must hold through M7.**

Acceptance criteria. Two lessons, grounded and ungrounded, indistinguishable in shape and distinguishable in evidence. **Report the §15.3 quality comparison** — first data on the §5.3 prediction.

M6 · Review UI and trust automation

Purpose. 100 statements reviewed in under an hour. Statements reach `AUTO_VALIDATED` with no human involvement.

Architectural objective. §26.3–26.6, §25.3, §20.3.

Files created.

```
src/app/api/knowledge/review/{queue,batch,decide}/route.ts
src/app/knowledge/review/page.tsx
src/server/knowledge/trust/{corroboration,contradiction,inference,promote}.ts
src/server/review/{merge,split}.ts
```

Implementation notes, binding.

- Corroboration counts **independent publishers, not documents**. Two NCERT editions are one source.
- **Contradiction blocks promotion** above `AUTO_VALIDATED`, checked synchronously inside the promotion transaction (ADR-9). This is the mechanism that makes human effort per statement *fall* as the graph grows.
- `DISPUTED` is a **suspension flag, not a rung** (ADR-8).
- **Split apportions observations by the `contextId` recorded on each.** Never guess. Source tombstone resolves as `AmbiguousSplit`.
- D2: similarity 0.85, **never auto-merge** (ADR-10).

Tests required. `trust-demotion` (contradiction demotes within one cycle) · corroboration counts publishers · contradiction blocks promotion · merge (loser id resolves forever, no reference breaks) · **split** (observations apportioned by context; source resolves ambiguous) · `split-resolvable`.

Failure modes. A wrong merge is far worse than a duplicate — silent, and every downstream record inherits it. Hence never auto-merge. Trust ladder gamed by volume → halt conditions plus publisher-level independence.

Rollback. Routes removable; the M3 CLI remains functional.

Acceptance criteria. Measured review rate ≥150/hr. A statement promotes to `AUTO_VALIDATED` untouched by a human, and a raised contradiction demotes it automatically.

M7 · Teaching Knowledge Layer ⚠️ GATE — the thesis test

Purpose. Lessons pre-empt misconceptions and use verified analogies.

Architectural objective. §13, §13.3, §13.5, §18C.

Files created.

```
src/server/knowledge/teaching/{asset,registry,select}.ts
src/server/knowledge/teaching/kinds/{misconception,analogy,workedExample}.ts
src/server/advisors/knowledge/extractAssets.ts
```

Database changes. `TeachingAsset`, `AssetEfficacy`. **Gated push.**

Implementation notes, binding.

- **ANALOGY without a `breakdownPoint` is rejected by schema (V14).** An analogy taught without its limit installs a misconception — the learner extends the mapping past where it holds.
- **Ship three kinds only:** `MISCONCEPTION`, `ANALOGY`, `WORKED_EXAMPLE` (D6). The registry accepts more; do not populate them.
- Consumers switch on **capabilities**, never on `kind` (§18C.1).
- `AssetEfficacy` is derived. No `quality` column anywhere.

Tests required. `analogy-breakdown` · `assetsFor` respects trust policy and context · `teaching.miss` emitted when no asset exists · lesson composition includes a misconception pre-empt when available.

Content. Author the three kinds for the M3 chapter only. 6–10 hours.

Rollback. `assetsFor` returns empty → M5 behaviour exactly.

⚠ **GATE.** Lesson quality versus M5, scored per §15.3.

- **M7 > M5 by ≥1.0 point** → thesis holds, proceed.
- **M7 ≈ M5 ≈ ungrounded** → **§5.3 is falsified. Stop.** Do not spend M9's 40–80 content hours. Reconsider the phase.

Acceptance criteria. A lesson names the “plants eat sunlight” misconception before introducing the equation.

M8 · Observation store

Purpose. Observations in a separate database; mastery computable as a query.

Architectural objective. §20.2, §17.1.

Files created. `src/server/observation/{record,mastery,efficacy,earn}.ts` · `prisma/observation.prisma`.

Database changes. A separate Postgres instance. `Observation` only.

Implementation notes, binding.

- `Observation.contextId` is **required** — it is what makes a future split resolvable.
- **No mastery table, no score column.** `mastery()` is a pure function of observations.
- `earn.ts` derives `REINFORCES` edges with `earned: true` from measured transfer.

Tests required. `store-separation` (no query joins the two stores) · mastery is pure · erasure (deleting a learner's observations leaves knowledge untouched) · an earned reinforcement edge appears.

Failure modes. Joining the stores — the failure that turns a DPDP erasure request into a knowledge-graph incident. Tested, not trusted.

Rollback. Stop recording. Knowledge unaffected.

Acceptance criteria. A `REINFORCES` edge exists with `earned: true` that nobody authored.

M9 · Assessment, breadth and releases

Purpose. Assessment items exist; every subject has a verified chapter; lessons replay exactly.

Architectural objective. §10, §19, C3, F5.

Files created. `src/server/knowledge/assessment/{item,registry}.ts` · `src/server/advisors/knowledge/extractItems.ts`.

Database changes. `AssessmentItem`, `ItemConcept` · **Gated push.**

Implementation notes, binding.

- Distractors carry `diagnosesMisconception`. A wrong answer that says *which* wrong model the learner holds is diagnostic; one that is merely wrong is a score.
- **F5 rule:** if the outcome is recorded as evidence it is an `AssessmentItem`; otherwise it is an `EXERCISE` teaching asset. Same prompt may be both, as different rows.
- No difficulty, no discrimination, no calibration (D5, L8).

Content order. Remaining Science → **Maths** (first real `PROCEDURE` workout) → Social Science → **English** (first real `SKILL` workout). 40–80 hours.

Tests required. Distractor diagnosis present · point-in-time replay (a lesson from three months ago resolves to exact statement and asset versions) · all three knowledge kinds represented and teachable.

Acceptance criteria. Every subject has one fully verified chapter — proving the model holds across fact, procedure and skill, which is the real test of §13.

13. Milestones and Phase Exit Reviews

13.1 Milestones

Milestone	Meaning	Phase
KM-1 Platform exists	store conformant, both implementations, walls enforced	M0
KM-2 Knowledge exists	one chapter verified, throughput measured	M3
KM-3 A student sees it	grounded lesson, every block traceable	M5
KM-4 Trust scales	<code>AUTO_VALIDATED</code> without a human; contradiction demotes	M6
KM-5 Teaching improves	misconception pre-empted; thesis tested	M7
KM-6 The loop closes	an earned reinforcement edge nobody authored	M8
KM-7 Breadth proven	all three kinds live across all subjects	M9

KM-3 is the milestone that matters. Everything before it is scaffolding. If KM-3 slips more than two weeks past plan, escalate — the program is drifting toward premortem cause 1.

13.2 Phase Exit Reviews

No phase advances automatically. Each ends with four reviews and a recorded Go / No-Go. On a solo program the reviewer is the same person wearing different hats on different days — the separation is what matters, not the headcount. Write the answers down; an unwritten review did not happen.

Architecture Review — one question: *does anything built in this phase contradict the frozen architecture?*

- ☐ Every new file sits in the module its boundary table (§7) assigns it
- ☐ No forbidden import introduced
- ☐ No stored conclusion added
- ☐ No delete path added
- ☐ Every deliverable traces to a named architecture section

Engineering Review — *is this code we can live with for three years?*

- ☐ No abstraction with exactly one caller
- ☐ No TODO that encodes a decision rather than a task
- ☐ Naming matches architecture terminology exactly
- ☐ Rollback verified, not asserted

Testing Review — *do the tests assert invariants or just exercise code?*

- ☐ Every new invariant has a named test
- ☐ No earlier test modified or deleted
- ☐ Failure fixtures exist, not only success fixtures
- ☐ The suite fails when the invariant is deliberately broken (verify by breaking it once)

Red Team Review — *how would I break this?*

- ☐ What input makes this produce wrong knowledge silently?
- ☐ What happens on an empty graph? A graph with one node? A cycle?

- ☐ What does a malicious source document achieve?
- ☐ What breaks if this phase runs twice?
- ☐ What breaks if this phase is interrupted halfway?

Go / No-Go. All four green → Go. Any red → the phase is not done. Gate phases additionally require their number reported (§15).

14. Testing Strategy

14.1 Three tiers

Tier 1 — standing invariants over data. Run in CI against the whole graph, forever. Load-bearing: `dependency-dag`, `composition-dag`, `reinforcement-cycles-pass`, `graph-conflict`, `grounding`, `provenance`, `trust-gate`, `trust-demotion`, `no-silent-uncertainty`, `analogy-breakdown`, `split-resolvable`, `identity`, `no-delete`, `store-separation`, `tenant-isolation`, `curriculum-independence`, `empty-platform`, `context-canonical`, `quote-never-served`.

Deleting one is abandoning the principle it guards (§2.5).

Tier 2 — determinism. Identical bytes → identical chunk ids. Fixed graph → identical closure order. Table-driven context matching. Snapshot path selection.

Tier 3 — conformance. `KnowledgeStore` correctness defined once, run against memory (fast, every unit test) and Postgres (fidelity, CI).

14.2 The golden set

One chapter hand-authored as ground truth, created during M3 while the reviewer already has the source open. Every extractor, prompt, chunk-size or model change is scored on precision, recall, grounding rate and duplicate rate.

Without it, *"the extractor improved"* is an opinion. With it, a number.

14.3 What is deliberately not tested

Model output quality (non-deterministic — the golden set measures it instead) · lesson aesthetics (human judgment, §15.3) · efficacy before observations exist.

15. Platform Success Metrics

Architecture KPIs, not business KPIs. Each has a definition precise enough to compute, a target, and a phase from which it is tracked.

15.1 Standing metrics

#	Metric	Definition	Target	From
K1	Graph integrity	count of <code>dependency-dag</code> / <code>composition-dag</code> / <code>graph-conflict</code> failures	0, always	M0
K2	Provenance completeness	statements above <code>MACHINE_PROPOSED</code> lacking provenance ÷ all such statements	0%	M3
K3	Grounding rate	proposals passing V3 ÷ proposals produced	≥85%	M2
K4	Ingestion throughput	chunks produced per minute, single process	≥200/min	M1
K5	Validation throughput	proposals fully validated per minute	≥100/min	M2
K6	Review throughput	see §15.2	≥150/hr	M3
K7	Duplicate rate	concepts with a ≥0.85-similarity sibling ÷ total concepts	<5%	M6
K8	Contradiction rate	open contradictions ÷ verified statements	<2%, none open >1 cycle	M6
K9	Search latency	p95 <code>resolve()</code>	<100 ms	M4
K10	Traversal latency	p95 <code>REQUIRES</code> closure, cache warm	<50 ms (ADR-12 falsifier)	M4
K11	Graph coverage	distinct topics taught grounded ÷ distinct topics requested	rising; no absolute target	M5
K12	Miss rate	<code>knowledge.miss</code> ÷ lessons started	falling	M5
K13	Asset coverage	concepts with ≥1 <code>MISCONCEPTION</code> ÷ verified concepts	≥40% for reviewed chapters	M7

K11 and K12 are the content backlog, ordered by real demand. They are the answer to *what should we verify next* — never a guess.

15.2 Review throughput — exact definition

Ambiguous in v1.0; this closes it.

K6 = statements reaching `OFFICIAL_SOURCE_VERIFIED` ÷ hours of active review, where active review is wall-clock time in the review tool with the session excluding any gap >5 minutes.

Rules: measured over a **single continuous session of ≥45 minutes** (shorter sessions overstate the rate — fatigue has not set in). Statements, not proposals — rejected proposals consume time and count against the rate. Batch-approved statements count individually. Report the number and the session length together.

15.3 Lesson quality — exact definition

This was the largest gap in v1.0. Both the M5 and M7 gates depend on comparing “lesson quality”, which was never defined — so neither gate could actually fire.

Protocol. Ten topics from the M3 verified chapter. For each, generate three lessons: **A** ungrounded (flag off), **B** grounded (M5), **C** grounded + assets (M7). Strip all evidence markers. Present blind and in shuffled order.

Scored 1–5 on four dimensions:

Dimension	1	5
Factual accuracy	contains a claim contradicted by the source	every claim traceable and correct
Explanatory quality	states facts	builds understanding; the <i>why</i> is present
Misconception handling	reinforces or ignores a common wrong model	names and pre-empts it
Coherence	blocks in arbitrary order	prerequisites precede what needs them

Score = mean across dimensions and topics. Reported to one decimal.

Decision thresholds.

Result	Meaning	Action
$B > A$ on accuracy, $B \approx A$ on quality	thesis on track — exactly the §5.3 prediction	proceed to M6/M7
$C > B$ by ≥ 1.0	thesis confirmed	proceed to M9
$C \approx B \approx A$	thesis falsified	stop before M9
$B < A$ anywhere	grounding is harming lessons	flag off; investigate before proceeding

Who scores. Tanveer, blind. A second scorer is better; with one scorer, score all thirty lessons in one sitting so the standard does not drift between sessions.

16. Performance Budgets

Engineering constraints. Exceeding one is a defect in the phase that introduced it, not a follow-up ticket.

16.1 Latency — teach path (the only user-facing path)

Operation	Budget	Measured	Breach action
<code>resolve(topic)</code>	p95 < 100 ms	M4	add rung-3 index
REQUIRES <code>closure</code> , cache warm	p95 < 50 ms	M4	ADR-12 falsifier → D10
REQUIRES <code>closure</code> , cache cold	p95 < 400 ms	M4	pre-warm on review commit
<code>selectPath</code> end to end	p95 < 200 ms	M4	profile before adding cache
<code>assetsFor(concept)</code>	p95 < 50 ms	M7	index on <code>(conceptId, contextId)</code>
Added latency to <code>startLesson</code>	< 300 ms p95	M5	flag off, investigate

The last row is the one that matters. Phase 1 streams the first block quickly; Phase 2 must not visibly delay it.

16.2 Latency — write and review paths

Operation	Budget
single proposal validation (V1–V15)	< 20 ms
promotion incl. synchronous contradiction check (ADR-9)	< 500 ms
batch commit, 8 statements	< 2 s
closure cache full rebuild after clear-all	< 30 s at 10 ⁵ concepts
review queue page load	< 1 s

16.3 Throughput

Operation	Budget
ingestion	≥200 chunks/min
extraction	model-bound; ≥1 chunk/s wall-clock with concurrency
validation	≥100 proposals/min

16.4 Storage and memory

Dimension	Budget	Note
memory store implementation	full conformance suite < 512 MB	it is a test fixture, not a product
single traversal	<code>maxNodes</code> ≤ 10,000 hard cap	no default (M0)
traversal depth	<code>maxDepth</code> ≤ 12; typical ≤ 6	deeper is a curriculum bug
<code>ClosureCache</code> row	< 256 KB serialized	exceeded ⇒ the closure is too wide
growth	monotonic, accepted (ADR-5)	10 ⁶ statements ≈ single-digit GB

16.5 Scale targets

Per architecture §21: 10⁴ concepts / 10⁵ statements / 10⁴ assets / 10⁶ observations, **with the closure cache**. Beyond that, D10's falsifier fires before anything is redesigned.

17. Operational Blueprint

The platform must be operable, not merely buildable.

17.1 Deployment

Aspect	Decision
Unit	the existing Next.js app; Phase 2 adds no service
Knowledge DB	same Postgres as Phase 1
Observation DB	separate Postgres instance from M8 (L6)
Schema changes	<code>prisma db push</code> , human-gated, never in CI
Order	schema first, then code — additive tables are backward compatible; code referencing a missing table is not
Rollout	flag-gated (§21.1); deploy dark, enable after verification

17.2 Monitoring

Built on the existing evidence spine — no new dependency (G6).

Signal	Source	Alert when
<code>knowledge.miss</code> rate	evidence events	>50% of lessons for 1 h
grounded lesson rate	evidence events	drops >20% in 1 h
extraction failure rate	<code>evidence/failure.ts</code>	>50% in one batch → halt source
promotion rate	<code>ReviewEvent</code>	zero for 7 days during a content phase
open contradictions	<code>Contradiction</code>	any open >1 review cycle
teach-path p95	evidence timings	>300 ms added (§16.1)
store health	<code>health/registry.ts</code>	either store unreachable

17.3 Logging

`log.ts` conventions carry forward. Every knowledge operation logs `conceptId`, `statementId`, `releaseId`, `PROMPT_VERSION`.
Never log `Provenance.quote` — L10 applies to logs, which are read by humans and shipped off-box.

17.4 Backup and restore

Store	Backup	RPO	RTO	Restore test
Knowledge	daily full + WAL	1 h	4 h	quarterly, mandatory
Observation	daily full	24 h	8 h	quarterly
Derived (closures, indexes)	none	—	—	rebuilt from source (ADR-11)

Derived data is deliberately not backed up. If it cannot be rebuilt, it was a stored conclusion.

17.5 CI/CD

Pipeline: `tsc --noEmit` → `lint` → `vitest run` → **Tier-1 invariants against a seeded graph** → `build`.

Tier-1 invariants run on every commit, not nightly. They guard properties that are cheap to check and catastrophic to lose.

Never in CI: `prisma db push`, `prisma migrate deploy`, any write to a production store.

17.6 Releases

A `Release` is an immutable snapshot of statement and asset versions. Lessons record their release id; replay resolves through it. Cut a release at every content phase exit.

18. Failure Playbooks

Each: detection, immediate action, recovery, prevention.

FP-1 · prisma db push fails mid-schema *Detect:* push errors; some tables exist, some do not. *Immediate:* **do not retry blindly.** Enumerate created tables. *Recover:* the push is additive and idempotent — fix the ordering defect (M0 FK order) and re-run. If a partially-created table has a wrong shape, drop that one table and re-push. *Prevent:* FK-ordered table list in §12 M0.

FP-2 · Graph corruption — a cycle appears in REQUIRES *Detect:* `dependency-dag` fails in CI with the cycle path printed. *Immediate:* **block deploys.** A cyclic prerequisite graph makes `selectPath` non-terminating or arbitrary. *Recover:* identify the offending edge from the printed path; append a demoting `ReviewEvent` (never delete). Re-run. *Prevent:* V9 forces human confirmation of every dependency classification — this is why that rule exists.

FP-3 · Trust failure — unverified content served to a learner *Detect:* `trust-gate` fails, or a lesson block resolves to a sub-`VERIFIED` statement. *Immediate:* `KNOWLEDGE_GROUNDING` off (§21.2). Teaching reverts to Phase 1 behaviour instantly. *Recover:* find the call site with the wrong `TrustPolicy`. Audit every call site, not just the one found. *Prevent:* ADR-6 — no default policy, omission fails to compile.

FP-4 · Ingestion failure — >50% of a batch fails validation *Detect:* extraction failure rate alert (§17.2). *Immediate:* **halt that source** (§12.5). Do not review the survivors — a source failing at that rate produces plausible-looking survivors too. *Recover:* inspect ten failures by hand. Parse defect → fix `ingest/`, re-chunk. Prompt defect → fix, bump `PROMPT_VERSION`, re-extract. Source unsuitable → deprecate it. *Prevent:* golden set catches extractor regressions before a real source runs.

FP-5 · Review corruption — a batch commits partially *Detect:* statements promoted without a matching `ReviewEvent`, or vice versa. *Immediate:* stop review. *Recover:* append compensating `ReviewEvent`s. **Never edit or delete existing events** — the audit trail is the recovery mechanism. *Prevent:* batch atomicity test (M3).

FP-6 · Search failure — `resolve()` returns nothing for known concepts *Detect:* K12 miss rate spikes with no content change. *Immediate:* not user-breaking — the miss path is a supported state and lessons still generate. *Recover:* check index existence first, alias table second, trust policy third (an over-strict `minimum` filters everything). *Prevent:* `empty-platform` and `curriculum-independence` tests.

FP-7 · A wrong merge is discovered *Detect:* a concept's observations or edges make no sense; mastery anomalies. *Immediate:* stop merging. *Recover:* **split** (M6), apportioning observations by recorded `contextId`. Observations whose context does not disambiguate resolve as `AmbiguousSplit` — they are marked, not guessed. *Prevent:* ADR-10 — never auto-merge. This playbook is why.

FP-8 · Migration failure — a push must be undone *Detect:* post-push tests fail. *Immediate:* the app is unaffected if code was not yet deployed (§17.1 order). *Recover:* drop added tables/columns. Additive-only means nothing existing was lost. *Prevent:* §20.2 rules — never rename in place, always nullable or defaulted.

19. Migration Strategy

19.1 Into the platform

Nothing to migrate. Agabi stores no knowledge. Every phase is additive: new tables, one new database, one changed call site.

19.2 Schema evolution rules

Rule	Reason
every <code>db push</code> is gated on a human (G7)	destructive changes must be seen
new columns are nullable or defaulted	existing rows must never break
never rename a column in place	add, backfill, deprecate
new types are registry inserts, never migrations	§33
new context dimensions are inserts	§18
tables created in FK order	FP-1

19.3 The fourteen migrations the architecture prevents

Listed in architecture §21.2. **Verify each remains prevented at every phase exit review.** If a phase introduces a need for one of them, the phase violates architecture and must be redesigned before merge.

20. Rollback Strategy

Phase	Rollback	Cost
P2	none needed (additive nullable columns)	—
M0	drop new tables	minutes
M1	delete directory	minutes
M2	disable pipeline entry point	minutes
M3	append demotion events	minutes; no data lost
M4	unused until M5	none
M5	flag off, or revert one line in <code>startLesson</code>	seconds
M6	remove routes; CLI remains	minutes
M7	<code>assetsFor</code> returns empty	seconds
M8	stop recording	seconds
M9	disable item extraction	minutes

The M5 rollback must remain true through M7. It is the program's safety net: if grounding harms lessons, teaching reverts instantly while the knowledge platform stays intact.

No phase requires a data migration to roll back. A consequence of append-only design, not an accident.

21. Feature Flags and Kill Switches

21.1 Flags

Flag	Controls	Default	Removed
KNOWLEDGE_GROUNDING	M5 bridge — grounded vs defaultOut-line	off until M5 acceptance passes	after M7 gate passes
TEACHING_ASSETS	M7 asset injection	off until M7 acceptance	after M9
AUTO_PROMOTION	M6 automatic promotion AUTO_VALIDATED	off until corroboration verified	never — permanent operational control

Flags are read once per request, never cached across requests. A flag that cannot be flipped without a deploy is not a kill switch.

21.2 Kill switches

Switch	Stops	Use when
KNOWLEDGE_GROUNDING=off	all grounded teaching	FP-3, or teach-path latency breach
AUTO_PROMOTION=off	machine promotion to AUTO_VALIDATED	corroboration or contradiction detection suspect
source deprecation	extraction from one source	FP-4
TEACHING_ASSETS=off	asset injection	a bad analogy reaches production

Every kill switch degrades to a working system, never to an error. Grounding off → Phase 1 lessons. Assets off → M5 lessons. Auto-promotion off → human-only review. This is the graceful-degradation property the architecture requires.

22. Risk Register

#	Risk	Prob	Impact	Detection	Mitigation	Phase
IR1	Content never populated	high	terminal	K11 flat	graceful degradation; miss-driven priority; M3 is one chapter end-to-end	M3
IR2	Review throughput below 150/hr	med	high	K6 at M3 gate	reorder M6 before M5	M3
IR3	<code>KnowledgeStore</code> leaks storage assumptions	med	high	second implementation fails conformance	memory implementation written first	M0
IR4	Silent conflation (one concept is really two)	med	high, silent	mastery anomalies, years later	split first-class; <code>contextId</code> on every reference; FP-7	M6
IR5	Silent duplication	med	high, silent	K7	aliases, dedupe, merge queue	M6
IR6	Prerequisite misclassified as reinforcement	high	high	learners blocked or mis-sequenced	V9 always human-confirmed	M2
IR7	Grounding does not improve lessons	med	strategic	§15.3 at M5/M7	falsifiable prediction; stop before M9	M7
IR8	Extractor produces grounded-looking nonsense	med	high	golden set, K3	V3 machine-checked, never model-judged	M2
IR9	Bad analogy installs a misconception	med	high	efficacy, late	mandatory breakdown point; <code>TEACHING_ASSETS</code> kill switch	M7
IR10	Copyright	low	terminal	—	written not copied; quotes never served; legal gate before launch	M3
IR11	DPDP / minors	low	terminal	—	separate store; <code>purgeUser</code> ; legal gate	M8
IR12	Three graphs re-unified by a refactor	low	high	<code>reinforcement-cycles-pass</code> starts failing	W6/W7 + the test	M0
IR13	Latency regression from graph reads	med	med	§16.1 p95	closure cache; flag off at breach	M5
IR14	Two engineers diverge on an ambiguity	med	med	code review	any ambiguity found is a defect in this blueprint — report it	all
IR15	Second <code>db push</code> blocked mid-program	med	med	—	batch schema changes per phase; never mid-phase	all

IR1 remains the most probable failure. No amount of engineering addresses it; only shipping M5 early and letting K12 drive review priority.

IR4 is the one to fear. Silent, compounding, invisible until mastery behaves inexplicably years later.

23. Technical Debt Register

Debt must be visible. Each item: what, why accepted, removal strategy, priority, trigger.

#	Debt	Class	Why accepted	Removal	Pri	Trigger to remove
TD1	Closure cache invalidation is clear-all (D3)	accepted	correct and trivially right; targeted invalidation is a bug farm	invalidate by affected subgraph	low	rebuild exceeds 30 s (§16.2)
TD2	Search stops at rung 3 (D9)	accepted	rungs 1–3 cover known-item lookup, which is the actual query pattern	add vector rung behind the same interface	low	rung-3 miss rate >20%
TD3	Single reviewer, no reputation (D4/D8)	accepted	reputation with n=1 reviewer is theatre	quorum + reputation	med	contributors > 1
TD4	Postgres recursive CTE for traversal (D10)	accepted	ADR-12	traversal engine as derived store	low	p95 > 50 ms cache-warm
TD5	Memory store duplicates Postgres semantics	structural, permanent	it is the mechanism that keeps the interface honest (§5.1)	never remove	—	—
TD6	Three asset kinds only (D6)	accepted	registry is open; populating more before efficacy data is guessing	populate more kinds	med	K13 ≥40% and efficacy data exists
TD7	Contradiction check is synchronous (ADR-9)	accepted	correctness over latency	move async only if promotion cannot be servable meanwhile	low	promotion p95 > 500 ms
TD8	No content authoring UI	accepted	reviewers edit; they do not author	authoring surface	low	reviewers request it twice
TD9	Evidence emitted, nothing consumes it visually	accepted	the spine is the contract; dashboards are Phase 3	X7	low	operator asks “what happened” and cannot answer
TD10	<code>Workspace.title/subject</code> push outstanding	must clear before M0	—	run it (P2)	high	now

TD10 is the only high-priority item and it is a precondition, not debt to carry.

Rule. New debt is added to this table in the PR that introduces it, with its trigger. Debt without a written trigger is not accepted debt; it is a defect.

24. Future Extension Points

Each future engine plugs into a named interface without modifying the platform. Listed with the interface, what plugs in, and what must not change for the plug to work.

#	Extension	Interface	Plugs in as	Preserved by
X1	Vector / semantic search	<code>KnowledgeStore.resolve()</code> ⁴	a derived store behind the same interface	ADR-6, D9
X2	Community contribution + review quorum	<code>ReviewEvent.actorId</code> + reputation	more actors, quorum rule in <code>promote.ts</code>	L2 — the human floor is already the extension point
X3	Adaptive teaching	<code>assetsFor(concept, context, policy)</code>	a selection strategy consuming efficacy	capability dispatch (§18C.1), not <code>kind</code>
X4	Mastery engine / memory	<code>observation/mastery.ts</code>	a richer pure function over the same rows	L8 — nothing stored, so nothing to migrate
X5	Recommendation	<code>selectPath</code> + mastery filter	a filter step after topological sort	<code>selectPath</code> takes no learner input today, by design
X6	Digital Twin	<code>Observation</code> stream	a consumer of the observation store	L6 separation; <code>contextId</code> on every row
X7	Analytics	evidence events	a consumer of the existing spine	taxonomy is stable and versioned
X8	New knowledge kinds	type registry	a registry insert	§33 — registry, never migration
X9	New context dimensions	<code>ContextDimension</code> table	a row	canonical hashing is dimension-agnostic
X10	New asset kinds	<code>teaching/registry.ts</code>	a registry entry + capability declaration	consumers switch on capability
X11	Second storage backend	<code>KnowledgeStore</code>	a third implementation	conformance suite already proves two
X12	Multi-curriculum / new board	<code>Program</code> + <code>Mapping</code>	rows, not schema	L7 — knowledge never references curriculum

The property that makes all of these cheap: knowledge is stored as evidence with opaque identity, no stored conclusions, and no curriculum coupling. Every extension above reads the same rows differently rather than needing different rows.

25. Architecture Validation Matrix

Full traceability. Owner is Tanveer throughout — a solo program; the column exists so it stays true when it stops being true.

Architecture decision	Phase	Files	Test	Evidence	Status
§11 three graphs (ADR-1)	M0	graph/{dependency,composition,reinforcement}.ts	dependen- cy-dag , composi- tion-dag , reinforce- ment-cy- cles-pass , graph- conflict	CI green + cycle fixture	pend- ing
§18A opaque identity (ADR-4)	M0	ids.ts , concept.ts	identity	no FK on slug	pend- ing
§17 storage abstraction	M0	store/*	confor- mance (x2 impls)	both green	pend- ing
§18B bound- ed traversal	M0	graph/traverse.ts	traversal- bounded	compile error on missing bound	pend- ing
§18 context canonicalisa- tion	M0	context/canonical.ts	context- canonical	hash stability	pend- ing
append-only (ADR-5)	M0	store/KnowledgeStore.ts	no-delete	no delete method exists	pend- ing
§12.2 span preservation	M1	ingest/spans.ts	span- preservation	quote → page after transforms	pend- ing
§12.1 determinism	M1	ingest/chunk.ts	determinism	byte-identical chunk ids	pend- ing
§8.1 trust boundary	M2	advisors/knowledge/*	trust-gate	RawProposal has no trust field	pend- ing
§14.1 ground- ing (ADR-3)	M2	trust/validators/grounding.ts	grounding	fabricated-quote fixture rejected	pend- ing
§14 V1–V15	M2	trust/validators/*	15 pass/fail fixture pairs	all green	pend- ing
§26.2 human floor (ADR-2)	M3	review/decide.ts	trust- gate , provenance	no promotion without ReviewEvent	pend- ing
§25.2 review ergonomics	M3	scripts/review-cli.mjs	—	K6 ≥150/hr	pend- ing
§15 trust poli- cy (ADR-6)	M4	knowledge/search.ts	trustPoli- cy omission fails compile	type error	pend- ing
§18B.3 deter- ministic sort	M4	knowledge/path.ts	path determinism	snapshot stable	pend- ing

Architecture decision	Phase	Files	Test	Evidence	Status
§16 curriculum independence	M4	<code>knowledge/curriculum.ts</code>	<code>curriculum-independence</code>	works with zero programs	pending
§8.2 one call site (ADR-7)	M5	<code>conversation/manager.ts</code>	grounded e2e, flag-off identity	one-line rollback verified	pending
§26.5 DISPUTED flag (ADR-8)	M6	<code>trust/policy.ts</code>	disputed never served	policy rejects	pending
§14.3 contradiction (ADR-9)	M6	<code>trust/contradiction.ts</code>	<code>trust-demotion</code>	demotes in one cycle	pending
§20.3 merge/split (ADR-10)	M6	<code>review/{merge,split}.ts</code>	<code>split-resolvable</code>	apportioned by <code>contextId</code>	pending
§13.3 break-down point	M7	<code>teaching/kinds/analogy.ts</code>	<code>analogy-breakdown</code>	schema rejects	pending
§18C capability dispatch	M7	<code>teaching/registry.ts</code>	no <code>kind</code> switch in consumers	grep clean	pending
§17.1 store separation	M8	<code>observation/*</code>	<code>store-separation</code>	no cross-store join	pending
§20.2 mastery as query	M8	<code>observation/mastery.ts</code>	mastery purity	no mastery table	pending
§19 releases / replay	M9	<code>knowledge/version.ts</code>	point-in-time replay	3-month-old lesson resolves	pending
§10 assessment (C3, F5)	M9	<code>assessment/*</code>	distractor diagnosis	<code>diagnoses-Misconception</code> present	pending

Status values: pending → in progress → **verified** (test green in CI). Nothing is “done” — only verified.

26. Engineering Checklists

26.1 Before starting any phase

- ☐ Previous phase's exit review recorded Go
- ☐ Architecture sections for this phase re-read
- ☐ Schema changes identified and batched into **one** push, FK-ordered
- ☐ Rollback stated in the PR description
- ☐ No new dependency required (else stop and ask)

26.2 Before opening a PR

- ☐ `npx tsc --noEmit` clean
- ☐ `npm run lint` clean
- ☐ `npx vitest run` — all green, **no earlier test weakened**
- ☐ `npm run build` clean (stop `next dev` first)
- ☐ No Prisma import outside `store/**`
- ☐ No stored conclusion introduced
- ☐ No delete path introduced
- ☐ Every new invariant has a named test
- ☐ Any new debt added to §23 **with its removal trigger**

26.3 Knowledge-integrity checklist (M3 onward)

- ☐ K1 graph integrity: zero failures
- ☐ K2 provenance completeness: 0%
- ☐ Zero promotions above `AUTO_VALIDATED` without a `ReviewEvent`
- ☐ K7 duplicate rate below 5%
- ☐ K8: no contradiction left open beyond one review cycle

26.4 Code review checklist

- ☐ Does this trace to a named architecture section?
 - ☐ Could this force a rewrite later?
 - ☐ Any abstraction with exactly one caller? (premature)
 - ☐ Does it store anything recomputable?
 - ☐ Is the rollback real, or aspirational?
 - ☐ Does it cross a boundary in §7?
-

27. Definition of Done

A phase is done when **all** hold:

1. Every file in its Files list exists and is committed.
2. Every test in its Tests list passes.
3. No earlier phase's test was modified or deleted.
4. `tsc`, `lint`, `vitest`, `build` clean.
5. Acceptance criteria demonstrated, not asserted.
6. Rollback documented and **verified at least once** for M5.
7. Deferred decisions took their default and the observation was recorded.
8. **All four exit reviews (§13.2) recorded Go.**
9. Gate phases have reported their number per §15.
10. §25 rows for this phase moved to **verified**.
11. New debt registered with a removal trigger.

Not done: “the code is written but the test is flaky.” “It works locally.” “The push is pending.”

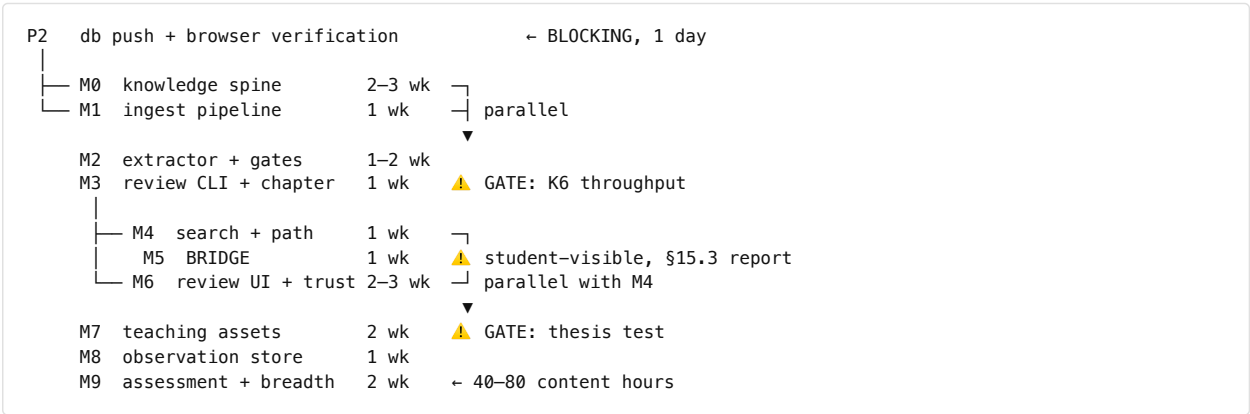
28. Final Readiness Review

Before M0 begins:

#	Check	Status
RR1	Architecture frozen at v1.0	✓
RR2	Evidence spine landed; replay proof passes	✓ 125 tests green
RR3	<code>prisma db push</code> for <code>Workspace.title/subject</code>	✗ BLOCKING
RR4	Multi-canvas verified in a browser	✗ blocked on RR3
RR5	Every phase traces to an architecture section	✓ §25
RR6	No circular implementation dependency	✓ §8.3
RR7	Critical path identified	✓ §9
RR8	Rollback exists for every phase	✓ §20
RR9	No new dependency required	✓
RR10	Gates defined with explicit stop conditions	✓ §15.2, §15.3
RR11	Gate measurement procedures defined	✓ §15.2, §15.3 — closed in v1.1
RR12	Kill switches defined and degrade gracefully	✓ §21.2
RR13	Performance budgets set	✓ §16
RR14	Failure playbooks written	✓ §18

RR3 and RR4 are the only blockers. One command and a browser session.

29. Build Order Summary



Engineering: ~13–15 weeks. Content: ~50–95 hours, concentrated in M9.

If forced to cut: M9’s breadth is compressible, M8 is deferrable, **M0 and M5 are not.**

30. Appendices

A · Standing rules

- 1. Never weaken an earlier phase’s test.
- 2. Never add a field that stores a conclusion.
- 3. Never promote above `AUTO_VALIDATED` without a human.
- 4. Never unify the three graphs.
- 5. Never join the knowledge and observation stores.
- 6. Never serve `Provenance.quote` to a learner — including in logs.
- 7. Never auto-merge two concepts.
- 8. When a deferred decision surfaces, take the default and record the observation.
- 9. Never add debt without a written removal trigger.
- 10. Never advance a phase without four recorded exit reviews.

B · Deferred decision defaults

#	Decision	Default	Resolved at
D1	extraction passes	four-pass	M3
D2	dedupe threshold	0.85, never auto-merge	M6
D3	closure invalidation	clear all on review commit	M6
D4	community quorum	expert-only	post-M6
D5	item calibration	store evidence only	Phase 3
D6	asset kinds	three	post-M7
D7	edge strength	authored, <code>earned=false</code>	Phase 3
D8	federated review	single reviewer	when contributors > 1
D9	vector search	rungs 1–3	when rung 3 fails
D10	traversal engine	Postgres	p95 > 50 ms with cache

C · Architecture traceability by phase

Phase	Architecture sections
M0	§9, §10, §11, §17, §18, §18A, §18B
M1	§12.1–12.3
M2	§8.1, §12.4, §14
M3	§19, §25, §26.2
M4	§15, §16, §18, §18B.3
M5	§8.2
M6	§14.3, §20.3, §25.3, §26.3–26.6
M7	§13, §18C
M8	§17.1, §20.2
M9	§10, §19, C3, F5

Every phase traces to at least one architectural decision. No phase exists without one.

D · Glossary

`AUTO_VALIDATED` — all applicable validators passed; no human involved; a real epistemic state, not a queue. `DISPUTED` — a suspension flag, not a ladder rung; trust level preserved. Grounded lesson — every block traces to a `VERIFIED` statement. `knowledge.miss` / `teaching.miss` — the content backlog, ordered by real demand. Golden set — one hand-authored chapter used to score extraction. Earned edge — a reinforcement edge derived from measured transfer, not authored. Derived artefact — rebuildable, never authoritative; not a stored conclusion (ADR-11).

End of Implementation Blueprint v1.1 — **FROZEN**.

Frozen alongside Architecture Baseline v1.0 as the authoritative pair for Backend Phase 2. Changes require a written amendment per the clause at the head of this document.